

タクトーン 質疑応答チートシート

チーム 23 — Arduino オーケストラ

2026-07-15 成果発表会 用

1 システム全体像

1.1 構成

ノード	ハードウェア	役割
指揮者 (node_01)	XIAO ESP32-S3 Sense + GY-521 (MPU6050)	IMU ジェスチャ認識, SoftAP 起動, UDP で CTRL/BEAT 配信
楽器 (node_02-05)	Arduino UNO R4 WiFi ×4	WiFi STA 接続, 時計同期, 楽譜進行, USB Serial で NOTE を PC へ送信
ドラム (node_06)	Arduino UNO R4 WiFi ×1	同上 (ドラム譜)
PC (各楽器に 1 台)	Mac + Processing 4	Serial 受信 → 加算合成で発音, ゲーム UI

1.2 データフロー

- 指揮者が IMU (5ms 周期) でジェスチャを取得し、振り下ろしから拍を検出
- 拍検出ごとに BEAT パケット (4 連送) を UDP マルチキャスト (239.0.0.1:5001) で全楽器へ配信
- CTRL パケットを 20 Hz (50ms 周期) で常時配信 (BPM, velocity, 状態, ゲーム情報)
- 楽器ノードは CTRL の timestampMs で時計同期し、playAtMasterMs の到達時刻で発音を予約
- 発音時刻に楽譜から NOTE パケット (20 B) を組み立て、USB Serial (115200 bps) で PC へ送信
- Processing が NOTE を受信し、JSON 楽器定義に基づく加算合成で発音

1.3 パケット仕様 (共通 20 バイト固定)

種別	type	方向	ペイロード概要
CTRL	1	指揮者 → 全楽器 (UDP)	bpmQ8 (BPM×8), velocity, state, mode, navCursor, targetBpm, score
BEAT	2	指揮者 → 全楽器 (UDP)	beatNo, playAtMasterMs (発音予約時刻)
NOTE	3	楽器 → PC (USB Serial)	partId, noteNumber, velocity, gate, durationMs, instrumentId
UI	4	楽器 → PC (USB Serial)	state, mode, navCursor, targetBpm, score, partId, bpmQ8

ヘッダ: magic 0x4F52 (ASCII “OR”), version 0x01, type, seq (単調増加), timestampMs (マスタ時刻) = 12 B. ペイロード 8 B. リトルエンディアン. `static_assert` で 20 B を保証.

2 Arduino — WiFi / UDP 通信

実装要点

- **トポロジ**: 指揮者が SoftAP (SSID OrchestraAP, パスワード orchestra2026, ch6) を起動. 楽器は STA で接続
- **プロトコル**: UDP マルチキャスト 239.0.0.1:5001. TCP は使わない (低遅延・ブロードキャスト不要のため)
- **BEAT 連送**: 同一 BEAT を **4 連送** (beatRedundancy=4), 各回 **2 ms** 間隔 (beatGapMs=2) で送信. radio 状態を変化させてまとめ落ちを軽減
- **CTRL 送信**: 20 Hz (50 ms 間隔). BEAT より後に送信 (playAtMasterMs のマージンが薄いため BEAT 優先)
- **発音予約**: playAtMasterMs = masterNow + 220 ms. SoftAP の DTIM バッファリングによるバースト配送 (~204.8 ms 周期) を吸収するための余裕
- **時計同期**: 楽器は CTRL の timestampMs を使い min フィルタ (窓長 2000 ms) で offset を推定. 窓内の最大 offset サンプル = 最小配送遅延のサンプルを採用. スナップ閾値 1000 ms で指揮者リセットに即追従
- **WiFi 再接続**: STA 側は isLinkUp() が false になると 2000 ms 間隔で自動再接続. 復帰時に udp.stop() → beginMulticast() でマルチキャスト購読を貼り直す
- **ライブラリ**: ESP32 は WiFi.h + WiFiUdp.h, UNO R4 は WiFiS3.h + WiFiUdp.h (#if defined(ARDUINO_ARCH_ESP32) で分岐)

Q. なぜ TCP ではなく UDP マルチキャストを選んだのか？

A. 1 台の指揮者から複数台の楽器へ同時配信するため. TCP はコネクション管理のオーバーヘッドと再送遅延があり, リアルタイム音楽同期には不向き. UDP はパケットロスの可能性はあるが, BEAT 4 連送で冗長化している.

Q. パケットロスはどう対処している？

A. BEAT は 4 連送 (2 ms 間隔) で radio 状態を変えながら送り, まとめ落ちを軽減. 楽器側は同一 beatNo の重複到着を検出し, 初到着分だけ発音予約に使い, 2 個目以降は時計同期の追加サンプルとして利用.

Q. 時計同期の方式は NTP ベースか？

A. NTP は使っていない. CTRL パケットの timestampMs (指揮者の millis() 値) と楽器の millis() の差分 (offset) を min フィルタで推定する独自方式. NTP の往復計測は不要で, 片方向の配信だけで同期できる. min フィルタは窓内の最大 offset (= 配送遅延が最小だったサンプル) を採用することで, バッファリング遅延の影響を排除する.

Q. 220 ms の遅延は体感で問題にならないか？

A. 120 BPM で約半拍の固定遅延. 全楽器に同一の遅延がかかるため, 楽器同士の同期は保たれる. 指揮者の振りと発音の間に一定のずれがあるが, 実測で聴覚上の違和感は報告されていない. ビーコン間隔の最適化で 70 ms 程度まで短縮できる見込み.

Q. SoftAP のチャンネルは固定か？

A. ch6 固定. 2.4 GHz の中間チャンネルで干渉が比較的少ない. 本番環境 (教室) では他の AP との干渉があり得るが, SoftAP なので周囲の AP に合わせたチャンネル選択は行っていない.

Q. 同時接続台数の上限は？

A. ESP32 SoftAP は最大 6 台 (設定値). 楽器 5 台で制限内.

3 Arduino — IMU モーション検出

実装要点

- **センサ:** GY-521 (MPU6050), I2C 接続 (アドレス 0x68), ピン D4(GPIO5)=SDA, D5(GPIO6)=SCL
- **設定:** 加速度 $\pm 4\text{ g}$ (感度 8192 LSB/g), ジャイロ $\pm 2000\text{ dps}$ (16.4 LSB/dps)
- **サンプリング:** 5 ms 周期 (200 Hz), 14 バイトバースト読み取り (加速度 3 軸 + 温度 + ジャイロ 3 軸)
- **LPF:** IIR ローパスフィルタ $\alpha = 0.10$ (時定数 $\approx 45\text{ ms}$)
- **動加速度:** $\text{dynNorm} = |\vec{a}_{\text{LPF}}| - g_{\text{calib}}$. キャリブレーション時の静止加速度ノルム ($\approx 1\text{ g}$) を引くことで姿勢非依存
- **キャリブレーション:** 起動後 WiFi 接続完了で Calibrating に遷移. 2 秒間 (400 サンプル) の加速度ノルム平均を静止ノルムとして保持

3.1 拍検出アルゴリズム (ゲート式ステートマシン + 早期発火)

1. **Idle:** $\text{dynNorm} \leq 1.20\text{ g}$ で待機. 積分は走らせない (ノイズ蓄積防止)
2. **Armed 突入:** $\text{dynNorm} > 1.20\text{ g}$ で Armed に遷移. 動加速度を二重積分して擬似経路長 sPathLen を計算開始
3. **早期発火:** $\text{sPathLen} \geq 0.20\text{ m}$ に到達した瞬間に拍を発火 (振り始めから $\sim 100\text{ ms}$). 1 Armed セッション中 1 回限り. 不応期 350 ms (上限 $\approx 170\text{ BPM}$)
4. **発火後も Armed 維持:** Idle に戻さないことで「発火 $\rightarrow$ 即 Idle $\rightarrow$ 再 Arm $\rightarrow$ 再発火」の連続発火ループを構造的に防止
5. **リリース判定 (デバウンス 40 ms):**
 - (a) $\text{dynNorm} < 0.20\text{ g}$ (絶対値: 完全停止)
 - (b) $\text{dynNorm} < \text{ピーク} \times 0.40$ (相対値: ピークアウト後の減衰)
 - (c) Armed 開始から 800 ms 経過 (タイムアウト保険)
6. **テンポ推定:** 拍間隔の EMA ($\alpha = 0.30$), 範囲 40–240 BPM にクランプ

Q. なぜ加速度のノルムで判定するのか? 軸指定ではダメか?

A. 軸指定だとセンサの取り付け角度で誤検出・取りこぼしが起きた. ノルムは姿勢に依存しないスカラー量なので, どの向きで振っても安定して検出できる. ただし重力に直交する振りは過小評価される ($\sqrt{1+a^2} - 1 < a$) という限界はある.

Q. 1.20 g の閾値はどう決めたか?

A. 仕様書の 1.8 g は重力込み前提で届かず, 0.8 g では小さい揺れで誤検出した. 実機で調整して中間の 1.2 g に決定. 取りこぼし・誤検出が出たら 1.0–1.4 の範囲で再調整する想定.

Q. なぜ早期発火 (振り始めで発火) を採用したか?

A. 振り終わり (リリース時) に発火すると体感遅延が大きく, 指揮者の振りと言がずれる. 擬似経路長 (動加速度の二重積分) が 0.20 m に達した時点で発火すると, 振り始めから $\sim 100\text{ ms}$ で音が出て体感的に同期する.

Q. 連続スイングで二重発火しないか?

A. 構造的に防止している. (1) 発火しても Armed を維持し Idle に戻さないで, 同じ振り中に再 Arm $\rightarrow$ 再発火するループが起きない. (2) 不応期 350 ms で物理的に速い連打も抑制. (3) リリース判定にデバウンス 40 ms を設けて一瞬の dynNorm の落ち込みによる誤リリースも防止.

Q. ジャイロは使っているか?

A. 加速度のみで拍検出. ジャイロデータは取得しているが, 現在のロジックでは判定に使っていない. メニューナビ

(横/縦の振り分け) は加速度ベースの重力推定 + 振り方向分解で実現している。

4 Processing — UI とシリアル通信

実装要点

- フレームワーク: Processing 4 + Minim ライブラリ (加算合成オーディオ)
- 描画フレームレート: 90 fps (`frameRate(90)`)
- オーディオ: ステレオ出力, サンプルレート 44100 Hz, バッファ 512 サンプル
- シリアル: USB Serial 115200 bps. `serialEvent()` コールバック (Serial スレッド) でバイト受信 → magic バイト走査でフレーミング → 20 B パケットを `ConcurrentLinkedQueue` に push
- パケット処理: draw スレッドで毎フレーム `drainPackets()` → NOTE (type=3) なら発音トリガ, UI (type=4) なら画面状態を更新
- フレーミング: magic バイト 0x52 0x4F を走査し 2 バイト一致で `inFrame=true`, 以降 18 バイト読んで 20 B パケット完成
- 役割自動判定: UI パケット (type=4) を受信 → メイン操作 UI, NOTE のみ + `partId ≠ 0x02` → アナライザ
- マスターリセット検知: UI パケットが 2000 ms 途絶えたら `uiState=ST_IDLE` に戻し全発音停止

4.1 画面構成

画面	表示内容
PortSelect	シリアルポート一覧 (USB フィルタ+スクロール対応, クリックで開閉)
Waiting	キャリブレーション中 / 待機中 / Fallback
Menu	モード選択 (自由演奏 / ゲーム), カーソル表示
FreePlay	演奏画面 (BPM 表示, パート別最終ノート, ログパネル)
GamePlay	ゲーム演奏画面 + メトロノームガイド (フェードアウトする視覚+音声ガイド)
Result	ゲーム得点表示 (0-100 点)
Analyzer	アナライザ画面 (パート別最終ノート表示)

Q. なぜ Processing を採用したのか？

A. Arduino UNO R4 WiFi 単体では音声合成の処理能力が足りない (44.1 kHz サンプリングの加算合成は浮動小数点演算が大量に必要). Processing は Java ベースで Minim ライブラリによる UGen ベースのリアルタイム音声合成が容易で, Serial ライブラリも標準搭載されている。

Q. シリアル受信でデータが壊れたらどうなる？

A. magic バイト走査 (0x52 0x4F) で毎回先頭を探索するため, 途中のバイト欠損・ノイズが入っても次の magic から自動復帰する. version チェック (0x01) も行い, 無効パケットは破棄する。

Q. なぜ ConcurrentLinkedQueue を使っているか？

A. Serial スレッド (Processing の `serialEvent`) と draw スレッドが別スレッドで動くため. ロックフリーのキューで競合を避け, 描画スレッドが毎フレーム drain する構造で安全にパケットを受け渡す。

Q. 複数ポートを同時に開ける理由は？

A. 楽器は 5 台 + ドラム 1 台で, 各 PC が 1 台を担当する運用だが, デバッグ時には複数台を 1 台の PC に繋いでテストできる. ポートごとに `PortConn` オブジェクトで受信バッファを独立管理している。

Q. frameRate 90 は高すぎないか？

A. パケット受信は Serial スレッドなので frameRate は描画と音声管理の更新頻度. 90fps にすることで drainPackets() の呼び出し間隔を ~11 ms に抑え, パケット滞留を減らしている. 音声処理の発音タイミングにも影響する.

5 音声解析・音色合成

実装要点

- **合成方式: 加算合成 (Additive Synthesis).** 各楽器音を倍音の正弦波の重ね合わせで生成
- **ピッチ:** MIDI ノート番号から基本周波数を算出: $f_0 = 440 \times 2^{(n-69)/12}$
- **倍音:** 各倍音の周波数 $f_k = f_0 \times r_k \times \sqrt{1 + B \cdot k^2}$. r_k は倍音比 (理想では k), B は非調和性係数 (inharmonicity)
- **ビブラート:** ピッチ変調 $\text{pitchMul} = 2^{(d \times g \times \sin \phi_{\text{vib}})/1200}$ (cent 単位). onset 遅延あり
- **トレモロ:** 振幅変調 $s \times (1 - d/2 + d/2 \cdot \sin \phi_{\text{trem}})$
- **エンベロープ:** ADSR (attack/decay/sustain/release) + JSON 定義のサンプルカーブ (sustaining 時はループ区間で反復)
- **ノイズ:** 事前生成のノイズテーブルをエンベロープで変調し加算 (アタックノイズの再現)
- **音色定義:** pc_app/production/orchestra_resynth/data/*.instrument.json. ファイル名昇順の index が instrumentId
- **ドラム:** 別エンジン (DrumEngine.pde) で打楽器専用の合成
- **同時発音:** 最大 24 (MAX_POLYPHONY=24), 超過時は最古のボイスを強制リリース
- **消音:** Processing 側が durationMs から自動消音. NoteOff は送らない (NoteOn のみ)

5.1 輪唱の実現方法

- 全楽器ノードが**同一の楽譜** (score_data.cpp) を持つ
- 声部ごとに headRestBeats を設定: node_02=0, node_03=8, node_04=16, node_05=24
- 楽譜の進行は「指揮者の拍番号 - 1 - headRestBeats」で index を算出
- headRestBeats 分の拍を休符として読み飛ばし, 声部ごとにフレーズ単位でずれて入る
- 周回は CANON_CYCLE_BEATS=56 (曲長 32 + 最終声部遅延 24) のサイクル窓で管理
- 曲: かえるのうた (C major, 32 拍 = 4 フレーズ). 4 声部が 8 拍ずつずれて重なる

Q. なぜ加算合成を選んだのか？

A. 楽器の音色を物理的な特性 (倍音構造, 非調和性, エンベロープ) から再合成できるため. サンプリング方式と違い, テンポに依存する音長やピッチの自由な制御が可能. JSON で音色パラメータを外部定義でき, 楽器変更もファイル差し替えて済む.

Q. 音色の JSON はどこから来たか？

A. sound_lab/ の analyzer.py で実際の楽器音の録音から分析・抽出した. 基音検出 (pyin), 倍音抽出 (FFT/STFT), ADSR フィッティング, 残差ノイズ分離の 9 段パイプラインで JSON を自動生成.

Q. 音階の精度はどの程度か？

A. MOP2 検証 (合成音の基音推定 vs 平均律) で全 24 音の平均 | 誤差 | 0.74 cent, 最大 1.91 cent. 基準 3.6 cent 未満を達成. 誤差の主因は harmonics[n=1].ratio $\neq$ 1.0 (ホルン -1.91, トランペット -0.87 cent).

Q. 同時発音 24 で足りるか？

A. 4 声部 + ドラムが同時に鳴る最大ケースでも 5-6 音程度. durationMs 自動消音で不要なボイスは即リリースされ

るため、24 は十分な余裕がある。

Q. 非調和性 (inharmonicity) とは？

A. 実際の楽器の倍音は理論整数倍からずれる。ピアノの弦の剛性が典型例で $f_k = k \cdot f_0 \cdot \sqrt{1 + B \cdot k^2}$ で高次倍音ほど高くずれる。金管楽器は B が非常に小さい (ほぼ調和的)。

6 JSON 楽器定義の仕組み

6.1 ファイルの場所と読み込みルール

- 楽器定義ファイルは `pc_app/production/orchestra_resynth/data/` に配置
- ファイル名は `0_trumpets.tweaked.instrument.json`, `1_horns.tweaked.instrument.json` のように先頭の番号で決まる
- Processing 起動時に `rescanInstruments()` が `data/` フォルダ内の全 `.json` をファイル名昇順ソートで読み込む (`AudioManager.pde:18--42`)
- ソート後の `index` がそのまま `instrumentId` になる (0 = トランペット, 1 = ホルン, 2 = フルート, 3 = オルガン, 4 以降 = ドラム各種)
- 楽器ノードが NOTE パケットで送る `instrumentId` フィールドが、この `index` に対応する

6.2 JSON の全体構造

production 版のトランペット定義 (`0_trumpets.tweaked.instrument.json`) を例に示す。

キー	型	意味・実際の値
<code>fundamental_hz</code>	float	分析元の基本周波数。トランペットは 523.4 Hz (C5)
<code>midi_note</code>	int	分析元の MIDI ノート番号。 72 (C5)
<code>note_name</code>	str	音名。 "C5"
<code>sustaining</code>	bool	持続音か否か。金管 = true (鍵を押す限り鳴り続ける)
<code>inharmonicity_b</code>	float	非調和性係数 B 。金管は 0 (ほぼ完全調和)
<code>harmonics</code>	array	倍音の配列。トランペットは 30 本
<code>envelope</code>	obj	全体の音量エンベロープ (ADSR + サンプルカーブ)
<code>modulation</code>	obj	ビブラート・トレモロのパラメータ
<code>noise</code>	obj	アタックノイズ
<code>attack_sample</code>	obj	アタック部の波形サンプル (3969 点 , 22050 Hz)
<code>name</code>	str	楽器名

6.3 harmonics (倍音配列)

各要素は 1 本の倍音を定義する。

フィールド	型	意味・例 (トランペットの第 1 倍音)
<code>n</code>	int	倍音番号 (1 = 基音)。 1
<code>ratio</code>	float	周波数比。理想は n だが実測ではずれる。 0.9995
<code>amp</code>	float	振幅 (0-1 の相対値)。 0.783
<code>phase</code>	float	初期位相 (ラジアン)。 -2.81
<code>env</code>	array	倍音ごとの時間変化カーブ。32 点の配列

`ratio` が整数倍からずれる分だけ音色にキャラクターが生まれる。MOP2 検証でトランペットの基音は `ratio=0.9995`

のため -0.87 cent のピッチ偏差が生じた。

6.4 envelope (エンベロープ)

フィールド	型	意味・トランペットの値
attack_sec	float	音の立ち上がり時間. 0.02 s
decay_sec	float	減衰時間. 0.075 s
sustain_level	float	持続レベル (0–1). 0.676
release_sec	float	離鍵後の消音時間. 0.15 s
values	array	サンプルカーブ. 1200 点
rate_hz	float	values の再生レート. 77.9 Hz (= 1200 点で約 15.4 秒ぶん)
loop_start_sec / loop_end_sec	float	持続音のループ区間

6.5 modulation (変調)

パラメータ	値 (トランペット)	効果
vibrato.rate_hz	5.5 Hz	ピッチが 1 秒に 5.5 回揺れる
vibrato.depth_cents	5 cent	振れ幅 ± 2.5 cent (= ± 0.025 半音)
vibrato.onset_sec	0.5 s	発音から 0.5 秒かけてビブラートが立ち上がる
tremolo.detected	false	トランペットにはトレモロなし

6.6 JSON 抜粋 (トランペットの冒頭部分)

```
{
  "fundamental_hz": 523.444, "midi_note": 72, "note_name": "C5",
  "sustaining": true, "inharmoniccity_b": 0,
  "harmonics": [
    { "n": 1, "ratio": 0.9995, "amp": 0.783, "phase": -2.8116,
      "env": [0.05, 0.12, ..., 0.95] },
    { "n": 2, "ratio": 1.9987, "amp": 0.8795, "phase": 1.3278,
      "env": [...] },
    { "n": 3, "ratio": 3.0018, "amp": 1.0, "phase": -0.0923,
      "env": [...] },
    ... // 全 30 倍音
  ],
  "envelope": {
    "attack_sec": 0.02, "decay_sec": 0.075, "sustain_level": 0.676,
    "release_sec": 0.15, "rate_hz": 77.925,
    "values": [0.0, 0.002, ..., 0.95], // 1200 点
    "loop_start_sec": ..., "loop_end_sec": ...
  },
  "modulation": {
    "vibrato": { "rate_hz": 5.5, "depth_cents": 5,
      "onset_sec": 0.5, "detected": true },
    "tremolo": { "detected": false }
  },
  "noise": { "level": 0, ... }
}
```

7 音が出るまでの仕組み（ゼロから解説）

7.1 そもそもデジタルで音を作るとは

スピーカーは**振動板の位置**で空気を押して音を作る。コンピュータは「今この瞬間、振動板をどの位置にするか」を数値（-1.0 から +1.0）で指定する。この数値 1 個が **1 サンプル**。

- **サンプルレート** = 1 秒間にサンプルを何個出すか。本システムは **44100 Hz**（CD と同じ）
- つまり毎秒 44100 個の数値を計算してスピーカーに送り続ける
- **バッファサイズ** = まとめて計算する単位。 **512 サンプル**（ ≈ 11.6 ms 分）

最も単純な音は**正弦波**（sin 波）： $s(t) = A \sin(2\pi f t)$

f = 周波数（Hz）， A = 振幅， t = 時刻（秒）。 $f = 440$ Hz なら「ラ」の音になる。

7.2 加算合成の原理 — 倍音を足す

実際の楽器の音は sin 波 1 本ではなく、**基音 + 複数の倍音**が重なった複雑な波形。加算合成はこれを逆に利用して、sin 波を何本も足し合わせて楽器音を再構成する方式。

$$s(t) = \sum_{k=1}^N a_k \sin(2\pi \cdot f_k \cdot t + \phi_k)$$

- N = 倍音の本数（トランペットは **30 本**）
- a_k = 第 k 倍音の振幅（JSON の `harmonics[k].amp`）
- f_k = 第 k 倍音の周波数（後述の非調和性つき）
- ϕ_k = 初期位相（JSON の `harmonics[k].phase`）

各倍音の周波数は： $f_k = f_0 \times r_k \times \sqrt{1 + B \cdot k^2}$

f_0 は弾きたい音のピッチ（MIDI ノートから算出）， r_k は JSON の `ratio`， B は `inharmonicb`。 $\sqrt{1 + Bk^2}$ の項が「高い倍音ほどわずかに高くずれる」非調和性を再現する。

7.3 NOTE パケットから音が出るまでの全経路

1. **楽器ノード → USB Serial**: 楽器ノードが NOTE パケット（20 B, type=3）を 115200 bps で送信
2. **Serial スレッド受信**: Processing の `serialEvent()` が magic バイト 0x52 0x4F を検出してパケットを組み立て、`ConcurrentLinkedQueue` に push
3. **draw スレッド drain**: 毎フレーム（90 fps $\approx$ 11 ms 周期）`drainPackets()` がキューからパケットを取り出す
4. **handlePacket**: type=3 (NOTE) なら `NoteEvent` を生成、`gate==1` のとき `triggerNote()` を呼ぶ（`orchestra_resynth.pde:160--162`）
5. **triggerNote()**: `instrumentId` から `modelForId()` で JSON 定義（`InstrModel`）を取得。同時発音が `MAX_POLYPHONY=24` に達していたら最古のボイスを `noteOff()`。 `ResynthVoice` を生成し `v.patch(out)` で Minim の出力チェーンに接続（`AudioManager.pde:112--131`）
6. **ResynthVoice.uGenerate()**: Minim が 512 サンプルごとに呼び出す。1 サンプルずつ以下を計算:
 - (a) `ampAt(tSec)`: エンベロープ値（ADSR or サンプルカーブ）
 - (b) **ビブラート**: `pitchMul = 2^{(d \times g \times \sin \phi_{vib}) / 1200}`
 - (c) 30 本の倍音ループ: $f = f_0 \times r_k \times \sqrt{1 + Bk^2} \times \text{pitchMul}$, `phase[k] += 2\pi f / sr`, `s += amp \times harmEnv \times \sin(phase[k])`
 - (d) 正規化: `s \times = harmNorm`（全振幅の合計が 1 を超えないように）

- (e) ノイズ加算（アタック部の息成分の再現）
- (f) トレモロ（振幅変調）
- (g) 最終振幅: $s \times \text{ampAt} \times \text{gain} \times 0.9$

7. **Minim ミキシング**: 同時発音中の全 ResynthVoice の出力を AudioOutput が合算
8. **OS 出力**: Java Sound API → OS のオーディオドライバ → DAC → スピーカー

7.4 エンベロープ（音量の時間変化）

音は「パッと出てすぐ一定」ではなく、時間とともに音量が変わる。

- **Attack**（立ち上がり）：無音から最大音量に達するまで。トランペットは **0.02 s**
- **Decay**（減衰）：最大から持続レベルに下がるまで。 **0.075 s**
- **Sustain**（持続）：鍵を押し続けている間の音量レベル。 **0.676**（最大の約 68%）
- **Release**（リリース）：鍵を離してから完全消音まで。 **0.15 s**

JSON の envelope.values (1200 点) は、この大まかな ADSR をさらに細かくした実測カーブ。コードは simpleADSR フラグで「4 値の単純 ADSR」と「実測カーブ」を切り替えられる (SynthVoice.pde:46--57)。持続音 (sustaining=true) は loop.start.sec から loop.end.sec の区間をループ再生する。

7.5 同時発音の管理（ボイス管理）

- 最大同時発音数: MAX_POLYPHONY = 24
- 各発音は ResynthVoice オブジェクト (Minim の UGen を継承)
- 新しい音が来るとき上限に達していたら、最古の未リリースボイスを noteOff() (強制リリース)
- 消音は Processing 側が自動管理: NOTE パケットの durationMs から millis() + durationMs を計算し、時刻到達で noteOff() → リリース区間を経て done=true → Minim 出力チェーンから unpatch() (AudioManager.pde:180--192)
- ミキシング: Minim の AudioOutput が全 UGen の uGenerate() 出力を単純加算して最終波形にする

7.6 具体例: C4 (ド) の NOTE が来たら

1. 楽器ノードが NOTE パケットを送信: noteNumber=60, velocity=100, durationMs=500, instrumentId=0 (トランペット)
2. triggerNote() が呼ばれ, modelForId(0) でトランペットの InstrModel を取得
3. オクターブ移調: brassOctaveShift(0) = +12 なので実際の MIDI = 60 + 12 = 72 (C5)
4. ゲイン: $\text{velocity}/127 \times \text{brassPartAmplitude}(0) \times \text{masterVolume} = (100/127) \times 0.20 \times 2.0 \approx 0.315$
5. ResynthVoice 生成: $f_0 = 440 \times 2^{(72-69)/12} \approx 523.3 \text{ Hz}$
6. 毎サンプル (1/44100 s ごと) 合成処理が走る:
 - 第 1 倍音: $f_1 = 523.3 \times 0.9995 \times \sqrt{1+0} \approx 523.0 \text{ Hz}$, amp=0.783
 - 第 2 倍音: $f_2 = 523.3 \times 1.9987 \approx 1046.0 \text{ Hz}$, amp=0.880
 - 第 3 倍音: $f_3 = 523.3 \times 3.0018 \approx 1570.2 \text{ Hz}$, amp=1.000 (最強)
 - … 第 30 倍音まで同様に
7. 0.5 秒後にビブラートが立ち上がり, 5.5 Hz で $\pm 2.5 \text{ cent}$ ピッチが揺れ始める
8. durationMs=500 経過で noteOff() → 0.15 s の二次減衰カーブで消音
9. done=true で Minim から切り離されメモリ解放

8 音の仕組みに関する Q&A

Q. 加算合成と FM 合成・サンプリング音源は何が違う？

A. **加算合成**: $\sin$ 波を足す方式. 倍音構造を直接制御でき, 楽器の物理的な音色特性を忠実に再現できる. 計算コストが高い (30 倍音 $\times$ 同時発音数の $\sin$ 計算が必要). **FM 合成**: キャリア $\times$ モジュレータの組み合わせで豊かな倍音を少ない計算で生成. 金属的な音が得意だが, 特定の楽器音の再現は感覚的な調整になる. **サンプリング**: 録音した波形をそのまま再生. 最もリアルだが, テンポや音長の自由な制御が難しく, データ量が大きい. 本システムは「録音から倍音構造を分析し, 再合成で自由に音長・ピッチを制御する」というハイブリッドなアプローチ.

Q. なぜ加算合成を選び FM 合成やサンプリングにしなかったのか？

A. (1) 楽器ごとの倍音構造を JSON で外部定義でき, 音色の変更がファイル差し替えだけで済む. (2) MIDI ノート番号と duration から任意のピッチ・長さの音を生成でき, テンポに追従する楽譜再生に適している. (3) 実楽器の録音を analyzer.py で分析して JSON を自動生成するパイプラインが構築できた. FM 合成はパラメータ調整が直感的でなく, サンプリングは音長をテンポに合わせて伸縮するのが難しい.

Q. 発音のレイテンシはどれくらいか？

A. NOTE パケットを受信してから ResynthVoice.patch(out) までの処理は 1ms 未満. ただしオーディオバッファ 512 サンプル (≈ 11.6 ms) の遅延が加わり, OS のオーディオドライバでさらに 1 バッファ分の遅延が発生する可能性がある. 合計で約 15–25 ms. 指揮者からの固定遅延 220 ms と合わせて, 振りから音が出るまで約 250 ms 程度.

Q. 音量・音割れ対策はどうしている？

A. (1) 各パートに専用の振幅係数 brassPartAmplitude (トランペット 0.20, ホルン 0.17, フルート 0.15, オルガン 0.50) を設定し, 楽器間の音量バランスを調整. (2) $\text{harmNorm} = 1 / \sum a_k$ で全倍音の合計振幅を正規化し 1.0 を超えないようにする (InstrModel.pde:59). (3) 最終出力に $\times 0.9$ を掛けてヘッドルームを確保 (SynthVoice.pde:120). (4) masterVolume (デフォルト 2.0) はキーボードの +/- で調整可能 (範囲 0.05–4.0).

Q. JSON ファイルを書き換えると何が変わる？

A. **harmonics の amp**: 特定の倍音を強く/弱くして音色が変わる. 例えば第 3 倍音 (3 倍の周波数) を強めると鼻にかかった音色になる. **envelope**: attack_sec を長くすると弦のような緩やかな立ち上がりになる. **vibrato**: depth_cents を増やすとビブラートが強くなる. **inharmonicity_b**: 0 以外にするとピアノのような非調和的な響きになる. JSON を差し替えて Processing を再起動するだけで反映される.

Q. 音色 JSON はどうやって作ったのか？ (具体的に)

A. sound_lab/analyzer.py の 9 段パイプラインで実楽器の録音から自動生成した: (1) 基音検出 (pyin アルゴリズム), (2) STFT で時間周波数分析, (3) 倍音の周波数とパワーを抽出, (4) 位相を測定, (5) ADSR エンベロープをフィッティング, (6) 各倍音の時間変化カーブを抽出, (7) ビブラート/トレモロの検出と定量化, (8) 残差からノイズ特性を分離, (9) JSON に出力.

Q. 倍音が 30 本もあるのは多すぎないか？

A. 音の高さ (周波数) によってはサンプルレートの半分 (ナイキスト周波数 = 22050 Hz) を超える倍音は自動的にスキップされる (if (f >= sr*0.5f) continue;, SynthVoice.pde:104). 高い音ほど有効な倍音が減るが, 低い音では 30 本すべてが音色に寄与する. 30 本で金管楽器の特徴的な倍音構造を十分にカバーできる.

Q. ドラムの音はどう出している？

A. ドラム (instrumentId ≥ 4) は加算合成とは別の DrumEngine.pde で処理する. キック/スネア/ハイハット/クラッシュの 4 種のドラム音色を JSON で定義し, attack_sample の実波形データを持つ場合はそれを再生, ない場合

はノイズベースの合成で生成する。加算合成の金管楽器とは完全に独立した発音パス。

Q. オクターブ移調 (brassOctaveShift) はなぜ必要？

A. 楽器ごとに適切な音域で鳴らすため。楽譜の MIDI ノートは C4 (=60) 付近の共通音域で書かれているが、トランペットとフルートは +12 (1 オクターブ上 = C5), オルガンは -12 (1 オクターブ下 = C3), ホルンは 0 (そのまま C4) に移調する (AudioManager.pde:91--99)。これにより各楽器が自然な音域で演奏できる。

9 ログ取得・デバッグ

実装要点

- **ファーム側:** SerialDebug.h マクロ (DBG_PRINTF / DBG_PRINTLN)。SERIAL_DEBUG ビルドフラグで ON/OFF
 - 指揮者: SERIAL_DEBUG=1 (拍検出のデバッグ情報を 200ms ごとにダンプ)
 - 楽器: SERIAL_DEBUG=0 (USB Serial は NOTE バイナリパケット専用。テキスト混在を避ける)
- **指揮者ダンプ:** 200ms 周期で時刻・状態・WiFi・IMU 値・dynNorm・ゲート状態・経路長・BPM・beatNo・シーケンス番号を 1 行出力
- **イベントログ:** 状態遷移・WiFi 接続変化・拍検出の発生時に即時出力 (dumpEdges)
- **Processing 側:** 構造化ログ (OrcLogger.pde): [HH:MM:SS.mmm] [ノード ID] [カテゴリ] メッセージ 形式で stdout + 画面リングバッファ (200 行)
 - カテゴリ: NOTE, UI, SERIAL, AUDIO, STATE, ERROR, SYSTEM, METRO
- **MOP テスト:** MopTest.h で MOP_TEST ビルドフラグ切替。MOP1 (拍検出精度), MOP4 (同期誤差), MOP5 (通信遅延), MOP7 (起動時間), MOP8 (CPU 使用率) を個別に計測出力
- **シリアルログ収集:** tools/verification/scripts/serial_logger.py で複数ポートを同時に開き、PC 側タイムスタンプ + ポート名を付与して保存。pyserial 使用
- **MOP 分析:** tools/verification/scripts/ 配下の Python スクリプト群でログを解析 → PASS/FAIL 判定 → グラフ PNG 出力

Q. デバッグ出力の ON/OFF はどう切り替えるか？

A. platformio.ini の build_flags に -DSERIAL_DEBUG=1 を設定する。0 にするとマクロが空展開になり、コードサイズも実行時コストも 0 になる。楽器ノードは NOTE バイナリを Serial に流すため、デバッグ時以外は必ず 0 にする。

Q. MOP (Measure of Performance) の計測方法は？

A. ファームの MOP_TEST フラグで計測コードをコンパイル時に選択する。例えば MOP_TEST=4 では楽器ノードが BEAT 受信 → 発音時刻の localMaster レンジを出力し、serial_logger.py で収集した後 analyze.py で集計する。通常ビルド (MOP_TEST=0) では計測コードは一切コンパイルされない。

Q. PC 側ログはファイルに保存されるか？

A. Processing の println() は stdout に出力され、processing-java --run で起動した場合はターミナルに表示される。ファイル保存は serial_logger.py 側で行う。Processing 画面内にもリングバッファ (200 行) でログパネルを表示している。

Q. ログからどんな問題を発見したか？

A. MOP5 検証で「SoftAP のマルチキャストが省電力バッファで 204.8ms 周期のバースト配送になっている」ことを発見した。これは BEAT の lookahead (当時 45ms) が構造的に不足していることを意味し、220ms に引き上げた根拠になった。

Q. CPU 使用率の計測はどうしている？

A. MOP_TEST=8 で loop() 内の入力/ロジック/出力の 3 フェーズの micros() 差を毎ループ出力する。指揮者ノードの 3 フェーズ合計は 5ms 以内に収まっており、5ms 周期の IMU サンプリングに間に合う。

10 横断的な質問

Q. なぜこの構成（指揮者 + 楽器 + PC 分離）にしたか？

A. (1) UNO R4 WiFi 単体では加算合成のリアルタイム演算ができない ($44.1\text{ kHz} \times \text{倍音 } 20 \text{ 本} \times \text{同時発音数の浮動小数点演算量}$)。 (2) 指揮者は WiFi AP を起動する必要がある、ESP32-S3 が必要。 UNO R4 は STA のみ。 (3) 役割を分離することでファームウェアの複雑性を下げ、各ノードが単一責務で動く Embedded-Module-Architecture (EMA) を実現した。

Q. EMA (Embedded-Module-Architecture) とは何か？

A. loop() を入力 → ロジック → 出力の 3 フェーズに分け、各フェーズを IModule の配列で回す設計パターン。モジュール間は SystemData 構造体だけを介してデータを受け渡し、直接の関数呼び出しを禁止する。これにより IMU/WiFi/LED 等を独立にテスト・差し替えできる。

Q. 苦労した点は？

A. (1) SoftAP のマルチキャスト配送が DTIM バッファリングで 200ms 周期のバーストになる現象の特定と対策 (lookahead 220ms 化 + min フィルタ化)。 (2) 拍検出の閾値調整 (1.8→0.8→1.2g の試行錯誤、二重発火防止のゲート構造)。 (3) 4 声輪唱のサイクル窓ロジック (最終声部が 1 周を終えるまで先頭声部は次の周回を始めない制御)。

Q. 今後の展望は？

A. (1) ビーコン間隔の最適化で固定遅延 220ms を 70ms 程度に短縮。 (2) 楽器台数の拡張 (現行 5 台 → 6 台以上)。 (3) ゲームモードの拡充 (複数曲選択、難易度設定)。 (4) Processing から Web ベース (WebAudio) への移行検討。

Q. 費用・部品はいくらか？

A. 主要部品: XIAO ESP32-S3 Sense (～1200 円) × 1, Arduino UNO R4 WiFi (～3500 円) × 5, GY-521 (～300 円) × 1, ブレッドボード・ジャンパ線 (数百円)。 PC は大学の Mac を使用。 合計は約 2 万円前後。

Q. 生成 AI はどう使ったか？

A. 設計判断は著者が行い、AI (Claude) との対話を通じて共同でコードを書いた。 EMA のアーキテクチャ決定、Arduino の制約に照らしたレビュー、拍検出の閾値候補の提案などに活用。 最終的な判断と動作確認は人間が行い、誤りの責任は著者にある。

Q. 同期精度の MOP4 の実測値は？

A. 3 回実測 (延べ 560 拍 × 5 ノード) で中央値 7ms, 平均 10.8ms, 最大 65ms。 設計目標 20ms 以内を中央値で達成。 尾部の大きな値はループストール (SoftAP バースト位相に起因) で 9.2% 程度発生。

Q. MOP5 (通信遅延) はどうか？

A. 受信遅刻率を当初の 45.4% から lookahead 220ms 化で 1.9–3.1% まで改善。「発音予約が成立する」ことを指標のスコープとして再定義した。